University of TEHRAN

# به نام خدا

عنوان) گزارش تمرین اول معماری کامپیوتر پیشرفته( طراحی پردازنده MIPS به شکل pipeline )


نام و نام خانوادگی اعضای گروه:

بهزاد جنتی(۸۱۰۱۰۳۰۹۸)

ابوالفضل عصاریان نژاد(۸۱۰۱۰۳۱۸۷)

# فهرست:

مقدمه.....۳

حل تمرین طبق خواسته ها.....۶

طراحی مسیر داده و واحد کنترل.....۷

# مقدمه)

در ابتدا لازم است به اجزای کلی پردازنده MIPS اشاره کنیم:

۱- واحد کنترل ( controller )

۲- واحد مسیر داده ( Data-path )

واحد کنترل در پیاده سازی single cycle به شکل ترکیبی ( combinational ) پیاده سازی میشود.

هحچنین تمامی دستورات به یک سیکل (clock cycle ) برای اجرا نیاز دارند جدای از اینکه دستور از چه نوعی است( مثلا دسترسی به حافظه mem نیاز دارد یا خیر) پس طول سیکل با زمان اجرای کندترین دستور معین میشود.(lw)

# (MIPS single cycle

در این نوع پیاده سازی طول سیکل ثابت است و برای دستورات با زمان اجرا کمتر هم باید همان طول سیکل را طی کنیم.

در اینجا دو حافظه داریم: ۱- ins-mem ۲- Data mem (جزئی از پردازنده نیستند و جدا پیاده سازی میشوند.)

در پایپلاین هیچ stage ای قابل رد کردن نیست و تمام دستورات تمام stage ها را طی میکنند حتی اگر مثلا دستور نیاز به mem نداشته باشد.

طول سیکلمون برابر با کندترین دستور است و دستورات باید با هم sync باشند.

در single cycle بین stage ها register بگذاریم به حالت پایپلاین تبدیل میشود.

سیگنالهای کنترلی به همراه دستور جلو میروند اینجوری نیازی نیست واحد کنترل حواسش به همه ی دستورات باشد.( با اضافه کردن رجیستر این مورد محقق میشود.)

شماره رجیستر مقصد هم باید به همراه دستور جلو برود( جهت جلوگیری از وقوع خطا) rt,rd

# (MIPS single cycle

همه دستورات پنج stage طول میکشند چه با یک مرحله کار داشته باشند یا خیر.

این پنج stage عبارتند از:

۱- IF: (instruction fetch) مرحله اول از اجرای هر دستور میباشد و با توجه به آدرسی که pc دارد به آن اشاره میکند دستور را از instruction memory میخوانیم.( واکشی دستور)

۲- ID: (instruction decode) در این مرحله با توجه به opcode و function نوع دستور مشخص میشود و واحد کنترل سیگنالهای کنترلی لازم جهت اجرای دستور را به Datapath ارسال میکند.

۳- ALU: (Arithmatic Logic Unit) در این مرحله با توجه به opcode دستور و سیگنال ارسالی از واحد کنترل ( select alu ) عملیات محسباتی روی ورودی ها انجام میشود.

۴- mem

5- WB: در این مرحله حاصل در رجیستر مقصد نوشته میشود.

# پیاده سازی پردازنده طبق صورت تمرین)

در ابتدا ما دستوراتی را که در صورت سوال تمرین به شکل Bold مشخص شده بود را پیاده سازی کردیم. برای آن منظور ما بخش های مختلف پردازنده را جدا کردیم و هر کدام را با نام های FETCH.V, DECODE.V,EXECUTE.V مشخص کردیم و همینطور یک فایل mips_elements.v ساختیم و تمام کانستنس های برنامه را در آن ریختیم.

نکته) تمرین در اپلیکیشن ModelSim انجام شده است و پسوند V در فایلها به این معنی است که به زبان وریلاگ نوشته شده اند.

برای پیاده سازی دستورات بولد شده نیاز بود اول طول رجیستر AluOperation را از ۳ بیت به ۶ بیت تغییر دهیم( AluOperation = پایه سلکت ALU )

اکنون هر کدام از دستورات پیاده سازی نشده را تعریف کردیم و در واحد کنترل سیگنالهای لازم جهت اجرا این دستورات را تعبیه کردیم.

# معرفی تمام فایلهای برنامه:

## ماژول fetch )

اولین مرحله برای اجرای هر دستور واکشی آن از instruction memory بر اساس مقدار pc که آدرس مربوط به دستور را نشان میدهد میباشد برای پیاده سازی این مورد در تمرین نیاز به یک ماژول fetch داریم.

در این ماژول ما از ماژول inst_memory نمونه ساختیم و با توجه به آدرس حافظه دستور، دستور را از حافظه واکشی کردیم.

نکته یادآوری) در تعریف ماژول زمانی که بعد از اسم ماژول پارامتر نداریم( داخل پرانتز) یعنی این ماژول پیاده سازی به شکل کامپوننت سخت افزاری ندارد ولی برای ما کار انجام میدهد.

D:/education/computer architecture/computer assignment/t3/fetch.v - Default

```
Ln#
1   module fetch (
2               input clk,
3               input rst,
4               input [31:0] address,
5               output [31:0] instruction
6   );
7
8
9
10    inst_memory InstMem (
11        .rst(rst),
12        .adr(address),
13        .instruction(instruction)
14    );
15  endmodule
```

# ماژول decode )

- دستور واکشی شده جهت دیکود به واحد کنترل ارسال میشود تا پس از دیکود مشخص شود چه نوع دستوری است.
- در این ماژول ابتدا ما از decoder نمونه ساختیم و قسمت opcode و function دستورات را از هم جدا کرده ایم. دقت کنید که در فایل datapath.v برای پایپلاین کردن پردازنده ما بعد از دستور fetch، instruction را که ۳۲ بیت نیز میباشد در رجیستری به نام IR_register نوشته ایم و در مرحله دیکود (ماژول دیکود) ما دستور را از این رجیستر خوانده و عملیات دیکود را انجام میدهیم.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
//for decode stage
  decode decodeStage (
      .clk(clk),
      .rst(rst),
      .IR_instruction(IR_instruction),
      .func(func),
      .opcode(opcode),
      .read_data1(read_data1_reg),
      .read_data2(read_data2_reg),
      .inst_extended(inst_extended),
      .mem_read_data_reg(mem_read_data_reg),
      .pc_adder(pc_adder),
      .out_cntrl_wb(out_cntrl_wb),
      .register_write_address_reg2(register_write_address_reg2)
  );
```

# ادامه دیکود:

در مرحله دیکود ما علاوه بر opcode و function، رجیسترهای مبدا و مقصد و immediate دستورات را نیز جدا کرده آنها را بر حسب نیاز از رجیستر فایل خوانده و برای انجام عملیات روی آنها آماده سازی میکنیم. در این مرحله ما علاوه بر دیکود از دو mux که یکی از آنها برای آدرس برگشت در مرحله writeback است و دیگری نیز برای استفاده در دستورات پرش استفاده میکنیم. آخرین ماژول ما در این مرحله sign_ext است که دیتای immediate ۱۶ بیتی را به ۳۲ بیت تبدیل میکند.

بعد از decodestage در datapath ما ابتدا دستور اکستند شده را در رجیستری مینویسیم( برای پایپلاین و به منظور استفاده در پرش) سپس یک رجیستر ساخته ایم به عنوان رجیستر کنترل اول که خروجی آن برای کنترل انتخابگر ALU( پایه select ) و انتخابگر rs,rt mux برای مرحله wb میباشد.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
// controller register one (has all of the control signals)
  register #(
      .WIDTH(14)
  ) reg_cntrl_ex (
      .clk(clk),
      .RegWrite(1'b1),
      .reset(rst),
      .write_data({AluOperation, Branch, RegDst, DataC, MemRead, MemWrite, MemtoReg, RegWrite}),
      .read_data(out_cntrl_ex)
  );
```

رجیستر بعدی که برای هدف پایپلاین طراحی شده است رجیستری است که دیتای اول خوانده شده از رجیستر فایل را در خود نگه میدارد(rs).

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
//register that holds the data that came from register file for pipeline
 register regfile_rd1_register (
     .clk(clk),
     .RegWrite(1'b1),
     .reset(rst),
     .write_data(read_data1_reg),
     .read_data(RD1_reg)
 );
```

رجیستر بعدی رجیستری است که دیتای بعد از mux انتخابی بین دیتای دوم خوانده شده از رجیستر فایل rt و immediate را ذخیره میکند.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
//register come from mux2 of rd2 and imm
 register alu_mux_register (
     .clk(clk),
     .RegWrite(1'b1),
     .reset(rst),
     .write_data(alu_input2),
     .read_data(alu_input2_reg)
 );
```

رجیستر بعدی در مرحله execute رجیستری است که دیتای دوم خوانده شده از رجیستر فایل را برای مرحله memory ذخیره میکند.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
//register that holds the data that came from register file for pipeline
 register reg_rd2_ex (
     .clk(clk),
     .RegWrite(1'b1),
     .reset(rst),
     .write_data(read_data2_reg),
     .read_data(RD2_reg)
 );
```

دو رجیستر آخر در این مرحله رجیسترهایی هستند که مقدار rt,rd را برای مرحله writeback در خود ذخیره میکنند.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
// register that hold rt for WB
 register #(
     .WIDTH(5)
 ) reg_rt_wb (
     .clk(clk),
     .RegWrite(1'b1),
     .reset(rst),
     .write_data(IR_instruction[20:16]),
     .read_data(RT_reg)
 );

 // register that hold rd for WB
 register #(
     .WIDTH(5)
 ) reg_rd_wb (
     .clk(clk),
     .RegWrite(1'b1),
     .reset(rst),
     .write_data(IR_instruction[15:11]),
     .read_data(RD3_reg)
 );
```

# ماژول execute)

در اینجا ما از ماژول alu نمونه برداری کرده و عملیات روی دیتاهای گرفته شده انجام میشود. ماژول alu ) در این ماژول ما دستوراتی که در فایل نیاز بود را اضافه کردیم اما به جای استفاده از خود opcode به صورت بیتی تمام آنها را در فایلopcode_and_inst.v تعریف کرده و از constant آن در case ماژول alu استفاده کردیم.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
Ln#
220  //for execute stage
221    execute executeStage (
222        .data1(RD1_reg),
223        .data2(alu_input2_reg),
224        .alu_op(out_cntrl_ex[13:8]),
225        .alu_result(alu_result),
226        .zero_flag(zero)
227    );
228
229
```

D:/education/computer architecture/computer assignment/t3/opcode_and_inst.v - Default

```
`timescale 1ns / 1ns

//ALU instructions
`define ALU_ADD 6'b100000
`define ALU_SUB 6'b100010
`define ALU_AND 6'b100100
`define ALU_OR 6'b100101
`define ALU_XOR 6'b100110
`define ALU_NOR 6'b100111
`define ALU_SLT 6'b101010  // set less than
`define ALU_SLL 6'b000000  // shift left logical
`define ALU_SLLV 6'b000100 // shift left logical variable
`define ALU_SRL 6'b000010  // shift right logical
`define ALU_SRLV 6'b000110 // shift right ligical variable
`define ALU_SRA 6'b000011  // shift right arithmatic
`define ALU_SRAV 6'b000111 // shift right arithmatic variable
`define ALU_LUI 6'b001001  // load upper immediate
`define ALU_BEQZ 6'b001011 // branch equal zero
`define ALU_ADDU 6'b100001 // add unsigned
`define ALU_SUBU 6'b100011 // subtract unsigned
`define ALU_SLTU 6'b101011 // set less than unsigned
`define ALU_BEQ 6'b111111  // branch equal

// instructions opcodes and functions operand
`define RT 6'b000000
`define addi 6'b001000
`define addiu 6'b001001
`define slti 6'b001010
`define sltiu 6'b001011
`define lw 6'b100011
`define lui 6'b001111
`define sw 6'b101011
`define beq 6'b000100
`define beqz 6'b000111
`define j 6'b000110
`define jal 6'b000011
`define jr 6'b001000
`define jalr 6'b001001
`define andi 6'b001100
`define ori 6'b001001
`define xori 6'b001001
`define bne 6'b001001
```

D:/education/computer architecture/computer assignment/t3/mips_elements.v - Default

```
//arithmatics logical unit (ALU)
module alu (
    input        clk,
    input [31:0] data1,
    input [31:0] data2,
    input [5:0] alu_op,
    output zero_flag,
    output reg greather_flag,
    output reg [31:0] alu_result
);

  always @(alu_op, data1, data2) begin
    alu_result = 32'b0;
    greather_flag = 1'b0;

    case (alu_op)
      `ALU_AND:  alu_result = data1 & data2;
      `ALU_OR:   alu_result = data1 | data2;
      `ALU_ADD:  alu_result = data1 + data2;
      `ALU_SUB:  alu_result = data1 - data2;
      `ALU_SLT:  alu_result = (data1 < data2) ? 32'b1 : 32'b0;
      `ALU_XOR:  alu_result = data1 ^ data2;
      `ALU_NOR:  alu_result = ~(data1 | data2);
      `ALU_SLL:  alu_result = data2 << data1[4:0];
      `ALU_SLLV: alu_result = data1 << data2[4:0];
      `ALU_SRL:  alu_result = data1 >> data2[4:0];
      `ALU_SRLV: alu_result = data1 >> data2[4:0];
      `ALU_SRA:  alu_result = data1 >>> data2[4:0];
      `ALU_SRAV: alu_result = data1 >>> data2[4:0];
      `ALU_LUI:  alu_result = data2 << 5'd16;
      `ALU_BEQ:   alu_result = (data1 == data2) ? 32'b1 : 32'b0;
      `ALU_ADDU: alu_result = $unsigned(data1) + $unsigned(data2);
      `ALU_SUBU: alu_result = $unsigned(data1) - $unsigned(data2);
      `ALU_SLTU: alu_result = ($unsigned(data1) < $unsigned(data2)) ? 32'b1 : 32'b0;

      `ALU_BEQZ:
      if ($signed(data1) > $signed(data2)) begin
        alu_result = 32'b0;
      end else alu_result = 32'b11;
    endcase
  end
  assign zero_flag = (alu_result == 32'b0) ? 1'b1 : 1'b0;
endmodule
```

# سایر رجیسترهای تعبیه شده در datapath )

بعد از مرحله execute در فایل datapath ابتدا رجیستری برای کنترل کردن سیگنالهای کنترلی در واحد پایپلاین طراحی میکنیم.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
// controller register two (has three bits of the control signals)
register #(
    .WIDTH(6)
) reg_cntrl_mem (
    .clk(clk),
    .RegWrite(1'b1),
    .reset(rst),
    .write_data({out_cntrl_ex[6], out_cntrl_ex[4:0]}),
    .read_data(out_cntrl_mem)
);
```

خروجی این رجیستر کنترلی سیگنال we حافظه دیتا و انتخابگر mux بین دیتای خوانده شده از حافظه و خروجی alu است.

رجیستر بعدی خروجی alu را در خود ذخیره میکند.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
//register that holds the data that came from ALU for pipeline
register reg_alu_out_mem (
    .clk(clk),
    .RegWrite(1'b1),
    .reset(rst),
    .write_data(alu_result),
    .read_data(alu_result_reg)
);
```

# ادامه)

رجیستر بعدی که ورودی آن رجیستری است که در مرحله دیکود مقدار rt را در خود ذخیره کرده است تا برای حافظه دیتا، این مقدار را ذخیره کند و دوباره آن را در خود ذخیره میکند. به منظور پایپلاین کردن این رجیستر گذاشته شده است.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
Ln#
258
259     //register come from register file but go to the two next stage in memory stage for pipeline
260     register reg_rd2_mem (
261         .clk(clk),
262         .RegWrite(1'b1),
263         .reset(rst),
264         .write_data(RD2_reg),
265         .read_data(RD2_reg2)
266     );
```

رجیستر بعدی رجیستری است که مقدار آدرس برگشت را در خود نگه میدارد.

# Memory stage

مرحله مموری در فایل datapath پیاده سازی شده است.

D:/education/computer architecture/computer assignment/t3/datapath.v - Default

```
Ln#
278
279   data_memory data_mem (
280       .clk(clk),
281       .rst(rst),
282       .mem_read(out_cntrl_mem[3]),
283       .mem_write(out_cntrl_mem[2]),
284       .adr(alu_result_reg),
285       .write_data(RD2_reg2),
286       .read_data(read_data_mem),
287       .out1(out1),
288       .out2(out2)
289   );
290
291   mux2_to_1 #32 data_mux_mem (
292       .data1(alu_result_reg),
293       .data2(read_data_mem),
294       .sel  (out_cntrl_mem[1]),
295       .out  (mem_read_data)
296   );
297
```

بعد از آن یک mux قرار دارد که بین خروجی alu و دیتای خوانده شده از مموری انتخاب میکند ( انتخابگر آن از رجیستر کنترلی که قبلا توضیح داده شد می آید.)

رجیستر بعدی رجیستر کنترلی سوم است که خروجی آن we رجیستر فایل و دیگر خروجی برای پرش استفاده میشود.

```
299   // controller register three (has 1 bit of the control signals)
300   register #(
301       .WIDTH(3)
302   ) reg_cntrl_wb (
303       .clk(clk),
304       .RegWrite(1'b1),
305       .reset(rst),
306       .write_data({out_cntrl_mem[5:4], out_cntrl_mem[0]}),
307       .read_data(out_cntrl_wb)
308   );
309
```

# ادامه

رجیستر بعدی رجیستری است که بین خروجی alu و دیتای خوانده شده از مموری انتخاب میکند( انتخابگر آن از رجیستر کنترلی که قبلا توضیح داده شد می آید.) تا برای writeback از آن استفاده کند.

```
//the output come from mux_2 memory output and alu output
register reg_mem_out_wb (
    .clk(clk),
    .RegWrite(1'b1),
    .reset(rst),
    .write_data(mem_read_data),
    .read_data(mem_read_data_reg)
);
```

آخرین رجیستر آدرس برگشت برای عمل writeback را در خود ذخیره میشود.

```
//register that holds the write register address
register #(
    .WIDTH(5)
) reg_wb_address2 (
    .clk(clk),
    .RegWrite(1'b1),
    .reset(rst),
    .write_data(register_write_address_reg),
    .read_data(register_write_address_reg2)
);
endmodule
```

# ماژول رجیستر)

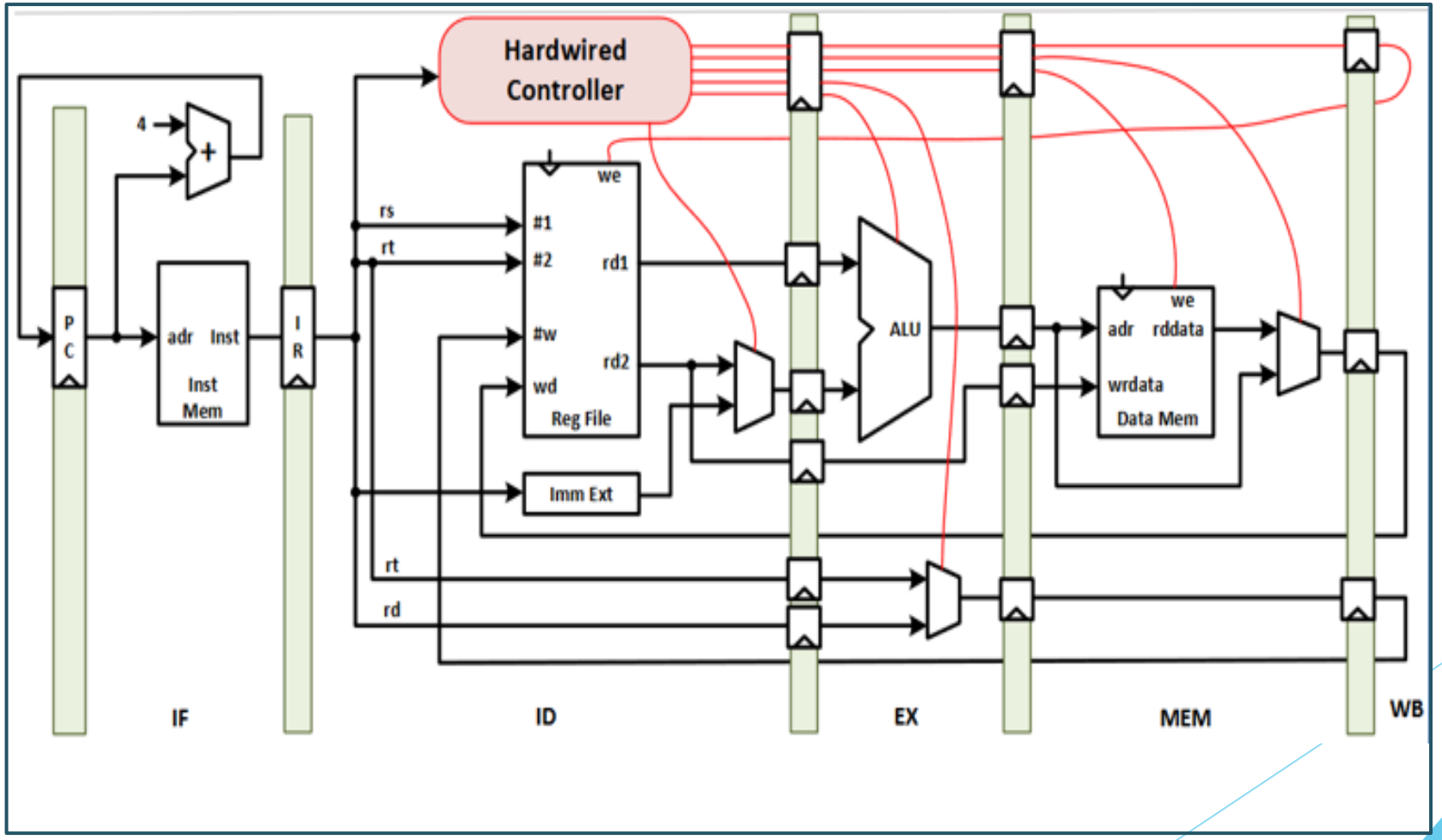
ماژولی است که برای ساخت رجیسترها از این ماژول نمونه سازی کردیم.

D:/education/computer architecture/computer assignment/t3/pipeline_register.v - Default

```
module register#( parameter WIDTH = 32)
(
        input clk,
        input RegWrite,
        input reset,
        input [WIDTH-1:0]write_data,
        output reg [WIDTH-1:0] read_data);

always@(posedge clk, reset)
        begin
                if(reset)
                        begin
                                read_data = 0;
                        end
                else if(RegWrite)
                        begin
                                read_data<=write_data;
                        end
        end
endmodule
```

# شمای استفاده شده برای طراحی پایپلاین:



# کد اسمبلی نوشته شده جهت تست پردازنده)

| | | |
|---|---|---|
| addi t4,t4,9 | 0 | |
| addi t5,t5,9 | 4 | |
| addi t30,t30,4 | 8 | → data mem از load کردن عدد از data mem ذخیره آدرس خانه دوم |
| addi t29,t29,36 | 12 | عدد 36 آدرس خانه دستور اول (مدیریت: برای loop2) → for loop2 → حلقه داخلی |
| addi t28,t28,20 | 16 | عدد 20 آدرس خانه دستور اول (مدیریت: برای loop1) → for loop1 → حلقه بیرونی |
| beq t4,t0, end | 20 | loop1 مقایسه |
| noop | 24 | |
| noop | 28 | → wb , mem , ex بی سرانجام → jump دست data Hazard سوگیری پیش‌بینی |
| noop | 32 | |
| beq t5,t0,end | 36 | → loop2 |
| noop | 40 | |
| noop | 44 | → wb , mem , ex بی سرانجام → jump دست data Hazard سوگیری پیش‌بینی |
| noop | 48 | |
| lw t1,offset(t31) | 52 | |
| lw t2,offset(t30) | 56 | |
| noop | 60 | |
| noop | 64 | read after write وابستگی داده‌ای انتهایی |
| noop | 68 | |
| noop | 72 | |
| noop | 76 | |
| sub t3,t1,t2 | 80 | |
| noop | 84 | |
| noop | 88 | |
| noop | 92 | → read after write وابستگی داده‌ای انتهایی |
| noop | 96 | |
| noop | 100 | |
| bgtz t3,t0, | 104 | → if t1 > t2 then Branch to 128 |
| noop | 108 | |
| noop | 112 | |
| noop | 116 | |
| sw t2,offset(t31) | 120 | |
| sw t1,offset(t30) | 124 | |
| addi t30,t30,4 | 128 | → t30 ← t30+4 → move to next number of row / no swap |
| addi t5,t5,-1 | 132 | → Counter 1 point decrease |
| noop | 136 | → data Hazard |
| jr t29 | 140 | → jump to Inner loop |
| noop | 144 | |
| noop | 148 | |
| addi t30,t31,4 | 152 | → end of loop2 (Inner loop) |
| noop | 156 | |
| addi t31,t31,4 | 160 | |
| addi t4,t4,-1 | 164 | |
| addi t5,t4,-1 | 168 | |
| jr t28 | 172 | → jump to loop1 |
| xxxx | 176 | → end of loop1 |

# توضیح کد اسمبلی نوشته شده در صفحه قبل)

این کد ۱۰ عدد را گرفته ( طبق صورت تمرین) و مرتب شده آنها را با استفاده از الگوریتم سورت حبابی تحویل میدهد. در ابتدا در متغیرهای t4 و t5 عدد ۹ که مقدار دو حلقه ی ما برای مرتب کردن است را ذخیره میکنیم.

سپس در رجیستر t30 آدرس خونه دوم حافظه دیتا را ذخیره میکنیم، برای load عدد از حافظه دیتا.

در رجیستر t29 عدد ۳۶ را ذخیره میکنیم که آدرس حافظه دستور است که بعدا برای پرش به آن نیاز داریم.

در رجیستر t28 عدد ۲۰ را ذخیره میکنم که آدرس حافظه دستور است که بعدا برای پرش به آن نیاز داریم.

سپس از دو دستور beq ( branch equal ) استفاده میکنیم تا مقدار رجیسترهای t4 و t5 را زیر نظر داشته باشم تا هرگاه مقدار آنها صفر شد از حلقه ها خارج شویم و برنامه خاتمه یابد. بعد از هر دستور beq از ۳ تا no op استفاده میکنیم تا دیتا هازارد پیش نیاید.

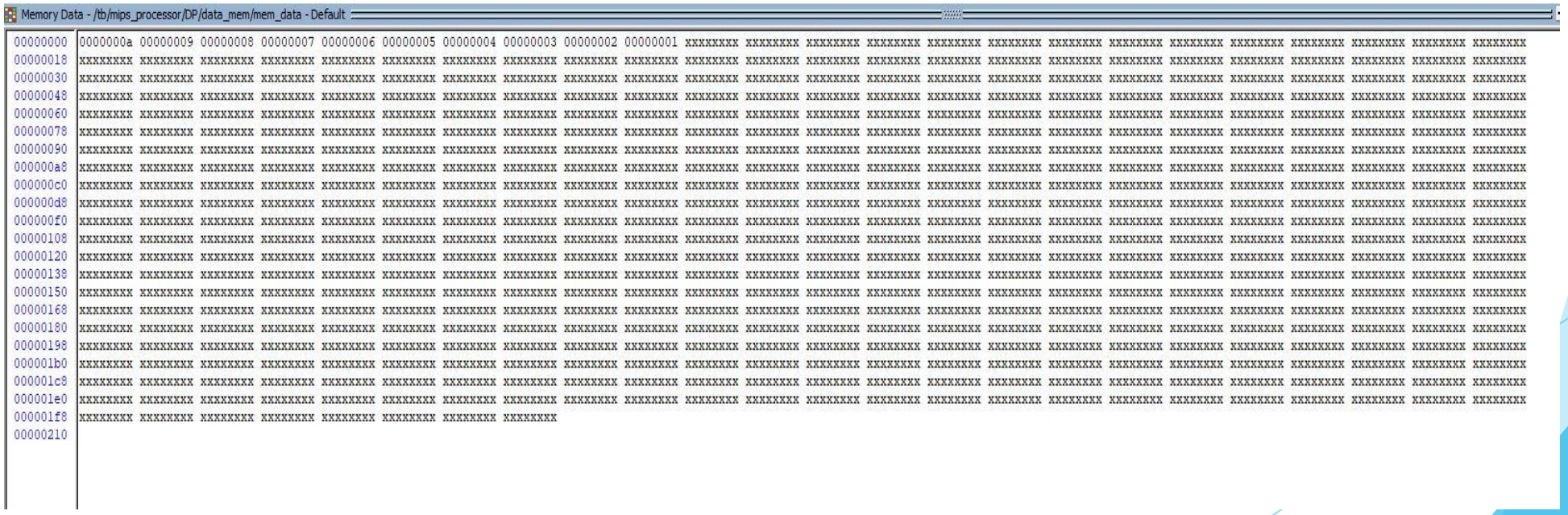
سپس با دو دستور load از خانه صفر حافظه (t31=0 ) و خانه یک حافظه (t30=4 ) در t1 و t2 ذخیره میکنیم.

سپس برای مقایسه دو عدد را از هم کم کرده و حاصل را در t3 میریزیم. سپس با دستور bgtz اگر مقدار t3 بزرگتر از صفر باشد به خانه ۱۲۸ پرش میکنیم در غیر این صورت مقدار pc جا به جا نمیشود و با دو دستور sw (store word ) مقدار دو رجیستر t1 و t2 برعکس آن چیزی که لود شده بودند در حافظه ذخیره میشود.

# ادامه توضیحات کد اسمبلی)

در هر دو حالت در خانه 128 ابتدا مقدار t30 چهارتا اضافه میشود تا خانه بعدی حافظه در حلقه بعدی لود شود و بعد از آن در خط بعدی از حلقه داخلی که مقدار آن t5 است یکی کم میشود.

سپس به خانه شماره 36 (t29) برنچ میکنیم و دوباره حلقه ها رو از اول پیمایش میکنیم. بعد از اتمام حلقه داخلی مقدار t30 برابر t31+4 میشود تا حلقه خارجی بار دیگر اجرا شود و دوباره حلقه داخلی به اندازه اعداد مرتب نشده تکرار شود.

در انتهای حلقه ی بیرونی باز با یک پرش به خانه 20 (t28 ) میرویم تا حلقه را بار دیگر تکرار کنیم. در انتها مقدار حلقه داخلی و خارجی با دستور addi با 1- جمع میشود تا حلقه ها به پایان برسند.

خروجی اعداد مرتب شده که جهت تست پردازنده در صورت سوال  مطرح شده بود به شکل زیر است: